

DELIVERY DECK

Seven2U

Interactive TV series companion. Never lose where you stopped. *Know what to expect* before watching.

DELIVERED • DOCKER COMPOSE UP → :7777

THE PROBLEM

Five series in rotation. *Which episode was I on?* Was that one worth it? Is the next one?

Trackers remember checkboxes. Ratings sites remember opinions.

Nobody tells you what an episode *is* before you press play.

THE SOLUTION

Search. Track. **Insight**. One surface, one port.

SEARCH

TVMaze, live

300ms debounce, AbortController, poster grid with rating and genres. Catalogue is never mirrored; 300s TTL cache.

TRACK

Watched + notes

Two Postgres tables. Optimistic toggle with rollback, per-season progress bars, notes on series or episode.

INSIGHT

Spoiler-free AI preview

Agno agent on Groq, 2.5s budget. Heuristic summary when the model is absent, labelled honestly.

Clean Architecture. *Domain owns the ports*, infrastructure implements them.

api/dependencies.py – composition root

The only place adapters are instantiated. Builds TVMazeGateway, SQL repos, GroqProvider; injects into services via FastAPI Depends.

api

routes, DTOs with `from_domain()`, domain error → 404/422/502

services

series · tracking · comment · insight. Constructor-injected Protocols. No HTTP, no SQL

infrastructure

TVMazeGateway (httpx+cache), SqlWatched/CommentRepository, GroqProvider (Agno), Heuristic

tests/fakes.py

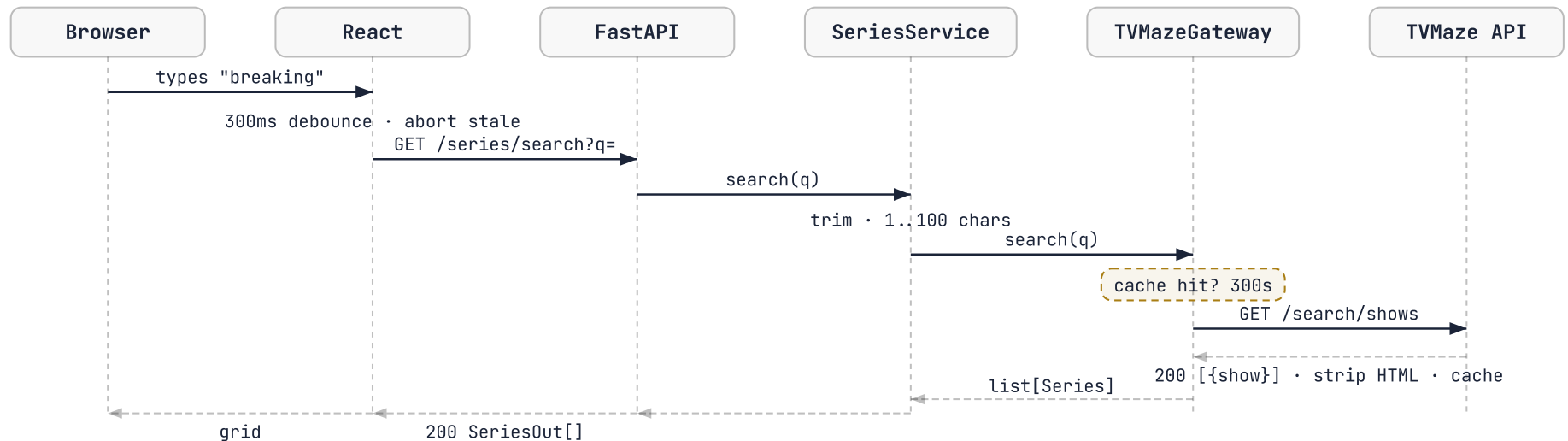
FakeCatalog, FakeWatched, FakeComment, StubProvider: drop-ins for every Protocol

domain

Frozen Pydantic models · Protocols `SeriesCatalog` · `WatchedRepository` · `CommentRepository` · `InsightProvider` · `group_by_season` · errors. Depends on nothing above it.

SEQUENCE · SEARCH

Debounce, abort, trim, cache. **Empty** and **error paths** are first-class.

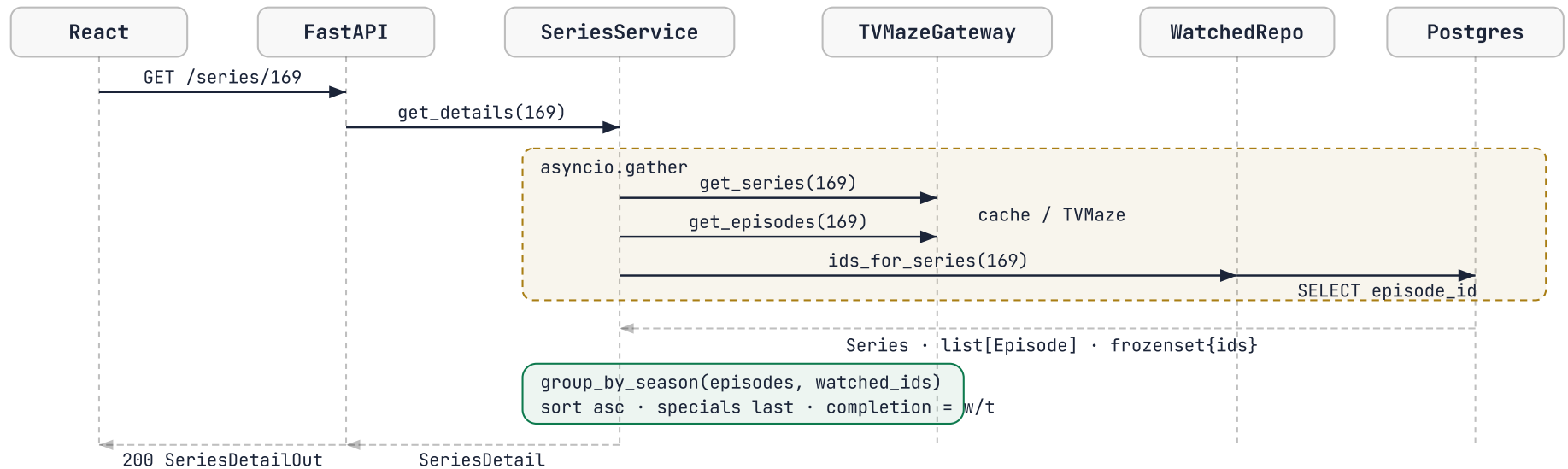


empty `q=""` never sent by client; server 422 `ValidationError`

no results 200 [] → `EmptyState` "No results found" **error** TVMaze 5xx/timeout → `UpstreamError` → 502 → `ErrorState` + `Retry`

SEQUENCE · SERIES DETAILS

Parallel fetch, **season grouping in the service**, progress from a pure rule.



Then, lazily and independently from the page:

InsightCard → GET /insights · CommentSection → GET /comments · Eye toggle → PUT /episodes/{id}/watched (optimistic, rollback + toast on error)

build InsightContext · try primary

no key → fallback directly

GroqProvider (Agnor)

```
Agent(model=Groq("qwen/qwen3.8-27b"))  
asyncio.wait_for(agent.arun(prompt), 2.5s)  
spoiler-free critic · ≤120 words
```

ok

SeriesInsight

source = "llm:groq"

timeout · error · empty

HeuristicTemplate

"heuristic:fallback"

no retry · no circuit breaker · never a 5xx

ISOLATION

InsightProvider

`generate_insight(ctx) → SeriesInsight`. Services see nothing else. New vendor = new file. Tests use a fake agent; zero network.

DATA MODEL

Two tables. Own *only* what the user creates.

```
-- idempotent by construction
CREATE TABLE watched_episodes (
  episode_id  INTEGER PRIMARY KEY,
  series_id   INTEGER NOT NULL,    -- indexed
  watched_at  TIMESTAMPTZ NOT NULL
              DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE comments (
  id          UUID PRIMARY KEY,
  series_id   INTEGER NOT NULL,
  episode_id  INTEGER, -- NULL = series note
  content     TEXT NOT NULL
              CHECK (length(trim(content)) > 0),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_comments_target
  ON comments (series_id, episode_id);
```

No FK to TVMaze. `TrackingService` checks the episode belongs to the series before writing. `PUT watched=true` twice returns the same `watched_at`; `false` on an absent row is a no-op.

DOCKER

`docker compose up --build` . **Port 7777. Zero CORS.**

STAGE 1

node:20-alpine

`npm ci && npm run build` → `dist/`

STAGE 2

python:3.13-slim

`pip install .`, copies `dist/` to `/app/static`. FastAPI mounts `/assets`, catch-all serves `index.html`: deep link `/series/169` reloads.

DB

postgres:16-alpine

Healthcheck, named volume, not published. `GROQ_API_KEY: ${GROQ_API_KEY:-}` optional on app.

Compose v3.3 `depends_on` does not wait for readiness, so the lifespan runs `wait_for_database` ($10 \times 1.5s$ `SELECT 1`) then `create_all`.
Observed on first boot: **attempt 1 failed, attempt 2 ready.**

TESTING

53 tests, under a second, *zero network*.

SUITE	#	ASSERTS	TOOL
Domain	6	Seasons sorted, specials last, foreign ids ignored, completion 0..1	pytest
Services	23	Blank query; episode $\notin$ series $\rightarrow$ 404; toggle idempotent; note trimmed; LLM $\rightarrow$ fallback on error/timeout	In-memory fakes
TVMaze gateway	6	Mapping, HTML strip, 404/5xx mapping, cache hit	<code>httpx.MockTransport</code>
Groq provider	5	Prompt shape, success, error, empty, timeout	Fake Agno agent
API	11	Status codes, DTOs, idempotency, 200-on-AI-failure	<code>ASGITransport</code> + real SQL repos on <code>sqlite+aiosqlite</code>

TRADE-OFFS

Every limitation is a *named next step*, not an oversight.

DECISION	BENEFIT	ACCEPTED LIMITATION
Single implicit user	No auth surface	Multi-user = <code>user_id</code> column
No catalogue mirror	Always fresh, two tables	Detail hits TVMaze (300s cache)
Idempotent <code>PUT</code> , not CRUD	Safe retries, easy optimistic UI	No watch history
Heuristic fallback, no retry/breaker	Simplest thing that never 5xx	Rule-based text without a key
Pydantic as domain	One model layer	Domain imports Pydantic
<code>create_all</code> , no Alembic	Zero migration tooling	Needed at first schema change
In-process TTL cache	No Redis	Per-replica

DEMO

`docker compose up --build`

→ **localhost:7777**

- 1 Search **Breaking Bad** → poster grid with rating and genre chips.
- 2 Open it → **5 seasons, 62 episodes**, progress bar per season.
- 3 Read the **Series insight** (violet card): `AI` badge with a Groq key, `smart summary` without.
- 4 Mark S01E01 watched → green flash, season 1 jumps to **14%**, survives a container restart.
- 5 Leave a note on the pilot → it feeds the episode insight context.

STATUS

Working solution

Verified on Docker with live TVMaze. Two bugs found by running it (shared-session gather, duplicate clear button), both fixed and committed.

NEXT

Extend in an afternoon

Auth = one column. New LLM vendor = one file. Migrations = Alembic. Shared cache = Redis.